



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر

ابزار دقیق

تمرین سری ۳

تاریخ تحویل: ۲۹ اردیبهشت ۱۴۰۵ | استاد درس: دکتر نیری

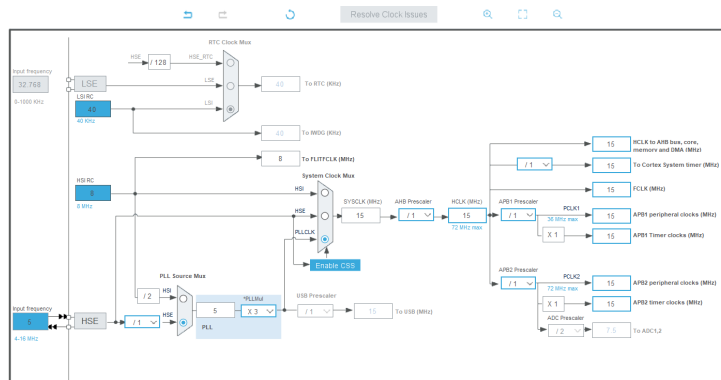
امیرمحمد میرحسینی - ۸۱۰۱۰۲۶۰۱

تنظیمات اولیه شبیه سازی

بر اساس دستورالعمل آزمایش، فرکانس کلاک سیستم باید بر اساس باقی مانده شماره دانشجویی بر عدد ۹ تعیین گردد. با توجه به شماره دانشجویی اینجانب (810102601)، محاسبات به شرح زیر است:

$$810102601 \pmod{9} = 1$$

بنابراین با توجه به باقی مانده به دست آمده، فرکانس کلاک سیستم برابر با 15 MHz تنظیم می شود.



شکل ۱: تنظیمات کلاک سیستم در محیط شبیه سازی

سوال ۱

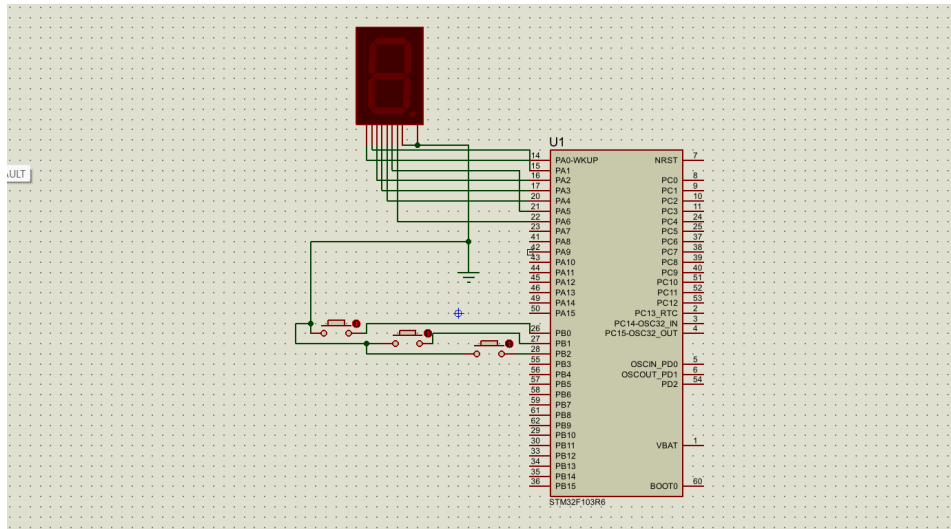
الف) طراحی سخت افزار و تخصیص پین ها:

- برای پایه های سون سگمنت، پین های PA0 تا PA6 در نظر گرفته شده اند.
- برای دکمه ها (به ترتیب دکمه معمولی و سپس دو دکمه وقفه)، پین های PB0 تا PB2 در نظر گرفته شده و تنظیمات آن ها در حالت PULL UP قرار گرفته است.

Search Signals							
Search (Ctrl+F)				☐ Show only Modified Pins			
Pin ...	Signal o...	GPIO o...	GPIO m...	GPIO P...	Maximu...	User La...	Modified
PA2	n/a	Low	Output ...	No pull-...	Low		☐
PA3	n/a	Low	Output ...	No pull-...	Low		☐
PA4	n/a	Low	Output ...	No pull-...	Low		☐
PA5	n/a	Low	Output ...	No pull-...	Low		☐
PA6	n/a	Low	Output ...	No pull-...	Low		☐
PB0	n/a	n/a	Input mo...	Pull-up	n/a		☒
PB1	n/a	n/a	External...	Pull-up	n/a		☒
PB2	n/a	n/a	External...	Pull-up	n/a		☒

شکل ۲: تنظیمات پین ها و وضعیت پایه ها در محیط نرم افزار

در محیط نرم افزار پروتئوس نیز پس از انجام تنظیمات اولیه CONFIGURE POWER RAILS، مدار را به صورت نشان داده شده در شکل زیر بستیم.



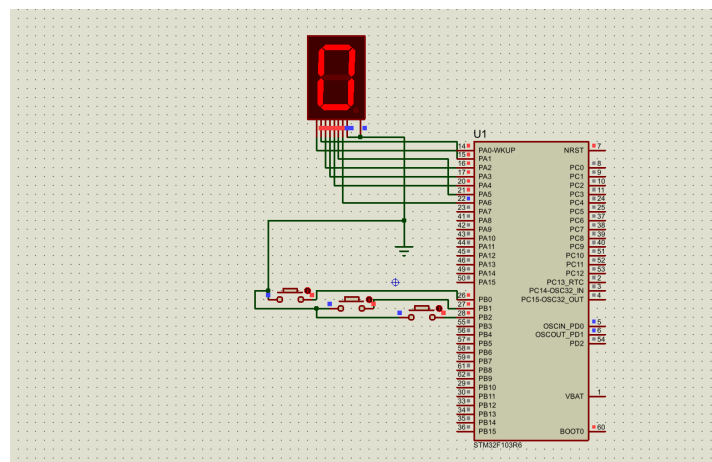
شکل ۳: شماتیک مدار بسته شده در نرم افزار پروتئوس

ب) نمایش وضعیت اولیه (عدد ۰):

طبق صورت برنامه ریزی، عدد ۰ باید بلافاصله پس از روشن شدن مدار و پیش از فشردن هرگونه کلیدی نمایش داده شود. به همین منظور، کدهای مربوط به این بخش دقیقاً قبل از حلقه اصلی (while(1) و در قسمت تنظیمات اولیه نوشته شدند تا تنها یک بار در زمان راه اندازی اجرا شوند.

از آنجا که سون سگمنت مورد استفاده از نوع **کاتد مشترک** است، برای روشن شدن هر سگمنت باید به پین مربوط به آن مقدار یک منطقی (SET) اعمال شود. بنابراین برای نمایش عدد ۰، پین های PA0 تا PA5 (سگمنت های A تا F) در وضعیت SET قرار گرفتند و پین PA6 (سگمنت G) در وضعیت RESET باقی ماند تا خط وسط خاموش بماند.

در نهایت با بارگذاری فایل hex. روی میکروکنترلر در نرم افزار پروتئوس، صحت عملکرد مدار و نمایش درست عدد ۰ در بدو استارت سیستم تأیید شد.



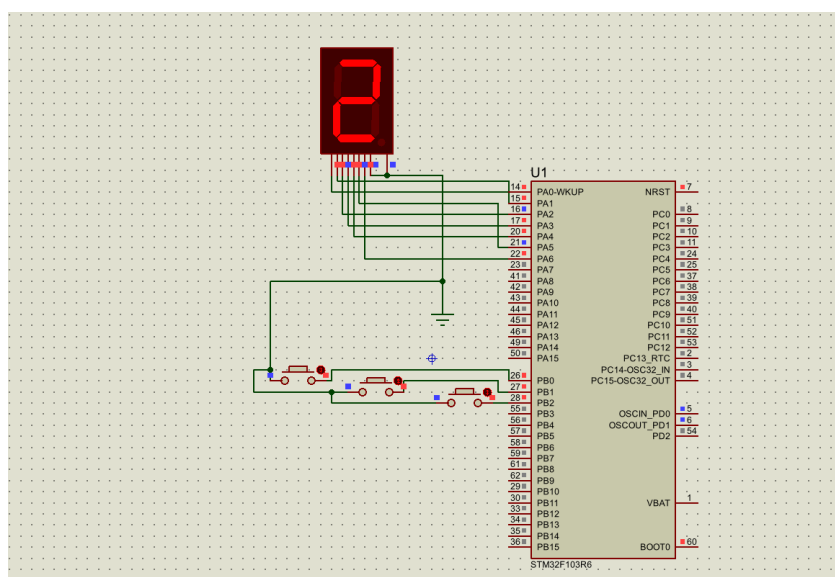
شکل ۴: نمایش وضعیت اولیه مدار و عدد ۰ روی سون سگمنت پس از اجرا

ج) حرکت پیوسته آسانسور (نمایش اعداد ۰ تا ۹):

در این بخش، هدف این است که با فشردن کلید استارت (PB0)، سیستم از حالت اولیه خارج شده و فرآیند نمایش طبقات آسانسور (اعداد 0 تا 9) به صورت پیوسته و با فواصل زمانی مشخص تکرار گردد.

جهت پیاده‌سازی این ساختار، به جای استفاده از حلقه‌های تو در تو که منجر به مسدود شدن فرآیندهای میکروکنترلر و سخت شدن کنترل برنامه می‌شود، از یک متغیر تحت عنوان `run_flag` استفاده شد تا برنامه به صورت (Non-blocking) اجرا شود. همچنین برای نمایش اعداد روی سون‌سگمنت کاتد مشترک، یک ماتریس دوبعدی از وضعیت پایه‌ها تعریف گردید. در راستای بهینه‌سازی مصرف حافظه RAM میکروکنترلر، متغیرهای مربوط به پرچم و شمارنده طبقات (i) به جای ساختار پیش‌فرض `int` از نوع `uint8_t` (یک بایتی) تعریف شدند تا از اشغال فضای اضافی حافظه جلوگیری شود.

منطق اجرای برنامه در حلقه اصلی به این صورت است که میکروکنترلر در ابتدا منتظر فشردن کلید متصل به پین PB0 می‌ماند. با فشردن این دکمه، مقدار `run_flag` برابر با یک می‌شود. سپس در هر دور از اجرای حلقه، وضعیت پین‌های پورت A بر اساس آرایه سگمنت‌ها به‌روزرسانی شده و پس از اعمال یک تأخیر زمان‌بندی نیم ثانیه‌ای (500 ms)، شمارنده طبقات یک واحد افزایش می‌یابد. در نهایت، هنگامی که شمارنده از عدد 9 عبور کند، مقدار آن مجدداً صفر می‌شود تا حرکت آسانسور به صورت پیوسته و چرخشی ادامه یابد.



شکل ۵: نمایش چرخه حرکت پیوسته آسانسور و شمارش اعداد روی سون‌سگمنت

د) تنظیم و پیاده‌سازی وقفه‌های اضطراری و تحلیل نتایج:

در این بخش، برای پیاده‌سازی الگوریتم توقف اضطراری، کدهای مدیریت کلیدها را از تابع اصلی (main) خارج کردیم و درون روتین وقفه استاندارد میکروکنترلر قرار دادیم.

منطق کار به این صورت است که به محض فشردن هر کدام از کلیدهای اضطراری، پردازنده از لوپ اصلی خارج شده و وارد تابع وقفه می‌شود. در ابتدای این تابع، فلگ شمارش (`run_flag`) را صفر می‌کنیم. با این کار، شرط حرکت در `while` اصلی برنامه غیرفعال شده و روند شمارش خودکار طبقات فوراً متوقف می‌شود.

سپس در ادامه تابع وقفه، با دو ساختار شرطی `if` و `else if`، پین تحریک‌کننده وقفه را بررسی می‌کنیم تا مشخص شود کدام دکمه فعال شده است:

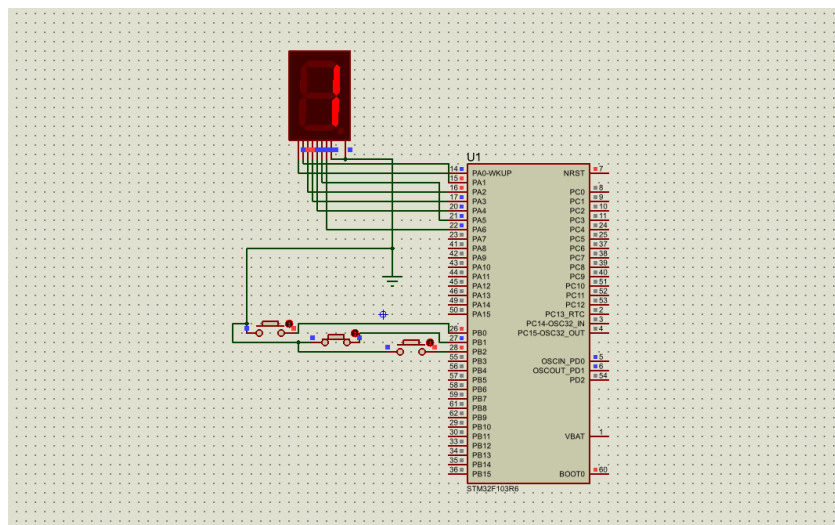
• اگر دکمه وقفه اول زده شده باشد، پین ورودی تشخیص داده شده، دیتای عدد 1 از ماتریس `segments` خوانده و روی نمایشگر ظاهر می‌شود.

• اگر دکمه وقفه دوم زده شده باشد، شرط دوم اجرا شده و دیتای عدد 9 روی خروجی سون‌سگمنت بارگذاری می‌شود.

نکته ابهام در سوال: با توجه به عبارت صورت سوال (که گفته «سیستم نباید به روند قبلی بازگردد»)، دو برداشت فنی وجود دارد. مشخص نیست که سیستم باید بعد از وقفه کاملاً روی حالت اضطراری قفل شود یا اینکه صرفاً حرکتش متوقف شده و آماده فرمان استارت مجدد باشد. به همین دلیل، پروژه را در قالب دو سناریوی مجزا پیاده‌سازی کردیم و دو فایل هگز خروجی گرفتیم:

۱. فایل هگز اصلی (Q1.hex): در این حالت، حلقهٔ قفل‌کنندهٔ while(1) را از انتهای روتین وقفه برداشتیم. سیستم بعد از نمایش عدد ۱ یا ۹، از وقفه خارج شده و در حالت استراحت قرار می‌گیرد. در این وضعیت اگر دکمهٔ شمارش دوباره فشرده شود، برنامه بدون نیاز به ریست سخت‌افزاری، شمارش را از همان نقطهٔ توقف شروع می‌کند.

۲. فایل هگز فرعی (Q11.hex): در این نسخه، یک حلقهٔ بی‌انتهای while(1) در انتهای بدنهٔ وقفه قرار دادیم. با این کار، میکروکنترلر بعد از وقوع وقفه و چاپ عدد، کاملاً در یک لوپ بسته قفل می‌شود و دیگر به هیچ فرمانی (حتی دکمهٔ شمارش یا استارت) جواب نمی‌دهد. در این سناریو، برای راه‌اندازی مجدد مدار، باید میکروکنترلر را سخت‌افزاری ریست کرد.



شکل ۶: فعال شدن دکمه دوم وقفه و نمایش عدد ۱ روی سون‌سگمنت

سوال ۲

بخش الف) تحقیق در مورد واحد TIM و تنظیم فرکانس PWM در STM32:

۱. واحد TIM و نحوه فعال سازی آن:
واحد TIM (تایمر) در میکروکنترلرهای STM32 یک شمارنده سخت افزاری مستقل است که بدون درگیر کردن هسته اصلی پردازنده، کارهایی مثل زمان بندی دقیق و تولید سیگنال PWM را انجام می دهد. برای فعال کردن این واحد در نرم افزار، ابتدا یکی از تایمرهای عمومی (مثل TIM2) را انتخاب می کنیم، کلاک سورس آن را روی Internal Clock می گذاریم و کانال خروجی مورد نظر (مثلاً Channel 1) را روی حالت PWM Generation تنظیم می کنیم تا پین مربوطه به صورت خودکار خروجی PWM شود. همچنین برای فعال سازی حالت Interrupt تایمر به صورت جداگانه می توان از بخش NVIC تیک آن را فعال کرد.

۲. پارامترهای PSC و ARR:

- **رجیستر PSC (Prescaler):** وظیفه این رجیستر، خرد کردن یا پایین آوردن فرکانس کلاک اصلی ورودی به تایمر است. چون فرکانس میکرو خیلی بالا است، با PSC سرعت شمارش را کم می کنیم تا تایمر سریع سرریز (Overflow) نشود. با نوشتن عدد در این رجیستر، کلاک اصلی بر $PSC + 1$ تقسیم می شود.
- **رجیستر ARR (Auto-Reload Register):** این رجیستر سقف شمارش تایمر را مشخص می کند. شمارنده تایمر از صفر شروع به شمردن می کند و به محض اینکه به عدد ARR رسید، دوباره صفر می شود و سیکل بعدی را شروع می کند. این رجیستر مستقیماً دوره تناوب و فرکانس موج PWM را تعیین می کند و مقدار واقعی آن برابر با $ARR + 1$ گام است.

۳. رابطه ریاضی فرکانس سیگنال PWM:

فرکانس نهایی موج PWM از رابطه زیر به دست می آید که در آن f_{clk} همان فرکانس کلاک ورودی به باس تایمر است:

$$f_{timer} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)}$$

۴. محاسبات برای تولید فرکانس 1 kHz:

با توجه به اینکه فرکانس کلاک ورودی تایمر در این پروژه روی 15 MHz (معادل 15,000,000 هرتز) تنظیم شده است، برای رسیدن به فرکانس هدف 1 kHz محاسبات را به این صورت انجام می دهیم:

$$1000 = \frac{15,000,000}{(PSC + 1) \times (ARR + 1)} \implies (PSC + 1) \times (ARR + 1) = 15,000$$

برای اینکه مقدار ARR یک عدد رند و مناسب برای تنظیم راحت دپوتی سایل (بین ۰ تا ۱۰۰ درصد) باشد، مقدار ARR را برابر با 999 فرض می کنیم (که یعنی 1000 گام شمارش از ۰ تا ۹۹۹):

$$(PSC + 1) \times (999 + 1) = 15,000 \implies (PSC + 1) \times 1000 = 15,000$$

$$PSC + 1 = 15 \implies PSC = 14$$

بنابراین تنظیمات نهایی تایمر در نرم افزار برای تولید فرکانس 1 kHz به این صورت به دست می آید:

$$\text{Prescaler (PSC)} = 14$$

$$\text{Counter Period (ARR)} = 999$$

بخش ب) راه اندازی درایور موتور L298، کنترل سرعت با ADC و تغییر جهت با وقفه:

۱. چرا از درایور سخت افزاری استفاده کردیم و منطق پایه ها چیست؟

پایه های خروجی میکروکنترلر STM32 جریانی در حد چند میلی آمپر تأمین می کنند که برای راه اندازی موتورهای DC اصلاً کافی نیست؛ حتی اگر موتور را مستقیماً به میکرو بنزیم پین های آن می سوزد. به خاطر همین از آی سی درایور L298 به عنوان واسطه قدرت در بخش (ج) استفاده می کنیم.

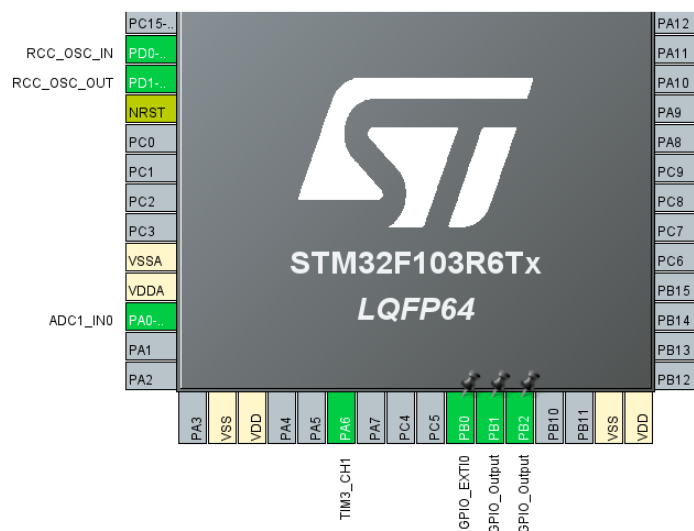
در این درایور، دو پایه دیجیتال IN1 و IN2 جهت چرخش را مشخص می کنند. به این صورت که وضعیت های معکوس (مثلاً $IN1=1$ و $IN2=0$) باعث چرخش موتور به یک سمت شده و با جابه جا کردن این دو وضعیت، جهت چرخش موتور معکوس می شود. پایه ENA (Enable) هم در سرعت موتور تأثیر می گذارد؛ یعنی سیگنال PWM را به آن وصل می کنیم تا با تغییر میانگین ولتاژ ورودی، سرعت موتور کنترل شود.

۲. منطق خواندن پتانسیومتر (ADC) و تناسب بندی سرعت:
پتانسیومتر یک ولتاژ آنالوگ بین ۰ تا ۵ ولت تولید می کند. (البته داخل پروتئوس، configure power rails و داخل ویدیو دیدیم که به ۵ ولت وصل می کنیم و پتانسیومتر رو ولتاژ پاور ۵ ولت هست اما در واقعیت باید به ۳.۳ ولت وصل بشه اما چون در پروتئوس برای شبیه سازی استفاده می کنیم موردی نداره.) واحد ADC میکرو که روی حالت ۱۲ بیتی و Continuous تنظیم شده، این ولتاژ را مدام نمونه برداری کرده و به یک عدد دیجیتال بین ۰ تا ۴۰۹۵ تبدیل می کند.
از طرفی، چون سقف رجیستر ARR تایمر ما در بخش قبل روی ۹۹۹ تنظیم شده، برای اعمال این سرعت به رجیستر مقایسه گر تایمر (CCR)، از فرمول تناسب خطی زیر در حلقه اصلی برنامه استفاده کردیم:

$$pwm_duty = \frac{adc_value \times 999}{4095}$$

این عدد خروجی (که بین ۰ تا ۹۹۹ تغییر می کند) مستقیماً پهنای پالس (Duty Cycle) سیگنال خروجی روی پایه ENA را تغییر می دهد و سرعت موتور را خیلی نرم و خطی بالا و پایین می کند.

۳. مدیریت دکمه تغییر جهت با روتین وقفه (EXTI):
برای اینکه پردازنده مدام در حلقه اصلی درگیر چک کردن پین دکمه نباشد، پایه دکمه را روی حالت وقفه خارجی با Falling تنظیم کردیم. با این کار، فرمان تغییر جهت به صورت وقفه و فقط در لحظه فشردن دکمه اجرا می شود. یک فلگ به نام direction_flag تعریف کردیم که با هر بار وقوع وقفه، وضعیتش نات (Not) می شود و وضعیت پایه های IN1 و IN2 را جابه جا می کند.



شکل ۷: وضعیت پایه ها در فایل ioc

۴. کدهای پیاده‌سازی شده در پروژه:

```

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    uint16_t adc_value = HAL_ADC_GetValue(&hadc1);
    uint16_t pwm_duty = (adc_value * 999) / 4095;
    TIM3->CCR1 = pwm_duty;
    HAL_Delay(10);

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */

```

شکل ۸: کد حلقه while

```

/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_1)
    {
        direction_flag = !direction_flag;

        if (direction_flag == 0)
        {
            // ON
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0, GPIO_PIN_SET); // IN1 = 1
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_1, GPIO_PIN_RESET); // IN2 = 0
        }
        else
        {
            // OFF
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0, GPIO_PIN_RESET); // IN1 = 0
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_1, GPIO_PIN_SET); // IN2 = 1
        }
    }
}
/* USER CODE END 4 */

```

شکل ۹: کد تابع وقفه

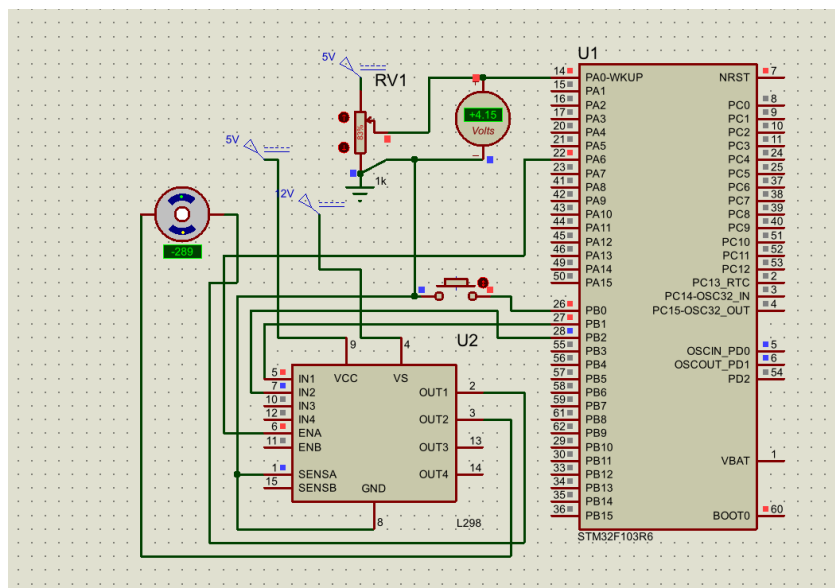
بخش ج) شبیه‌سازی سخت‌افزاری مدار در نرم‌افزار پروتئوس (Proteus):
در این بخش سیم‌کشی‌ها را به این صورت انجام دادیم:

۱. سیم‌کشی بخش ورودی آنالوگ (پتانسیومتر):
پتانسیومتر وظیفه تولید ولتاژ متغیر برای واحد ADC را بر عهده دارد. پایه بالای پتانسیومتر را به یک DC Generator با ولتاژ ۳/۳ ولت متصل کردیم. پایه پایین پتانسیومتر به زمین (GND) وصل شد و پایه وسط آن (پین خروجی متغیر) مستقیماً به پایه PA0 میکروکنترلر (کانال ۰ واحد ADC1) متصل گردید.

۲. سیم‌کشی بخش وقفه دکمه (تغییر جهت):
دکمه وظیفه تحریک وقفه خارجی برای تغییر فلگ جهت را دارد. یک پایه دکمه به پایه PB0 میکروکنترلر (پین EXTIO) و پایه دیگر آن مستقیماً به زمین (GND) وصل شد.

۳. سیم‌کشی آی‌سی درایور L298 و موتور دی‌سی:
آی‌سی درایور وظیفه دریافت فرمان‌های منطقی از میکرو و تأمین جریان موتور را دارد.

- پایه‌های فرمان دیجیتال: پایه IN1 درایور به پین PB1 میکرو و پایه IN2 درایور به پین PB2 میکروکنترلر متصل شدند تا جهت چرخش پل H را کنترل کنند.
- پایه کنترل سرعت: پایه ENA درایور مستقیماً به پین PA6 میکروکنترلر متصل شد (این پین همان خروجی سیگنال PWM کانال ۱ از تایمر ۳ است که در بخش‌های قبلی تنظیم کرده بودیم).
- پایه‌های تغذیه درایور: پایه GND به زمین مدار، پایه VSS (تغذیه بخش منطقی) به پاور +5V و پایه VS (تغذیه بخش قدرت موتور) به منبع ولتاژ +12V متصل شدند.
- پایه‌های خروجی: پایه‌های OUT1 و OUT2 درایور مستقیماً به دو سر موتور DC متصل شدند.



شکل ۱۰: شماتیک کامل مدار کنترل سرعت و جهت موتور DC در نرم‌افزار پروتئوس

سوال ۳

طراحی سیستم down-counter ۱۵ ثانیه‌ای با وقفه تایمر:

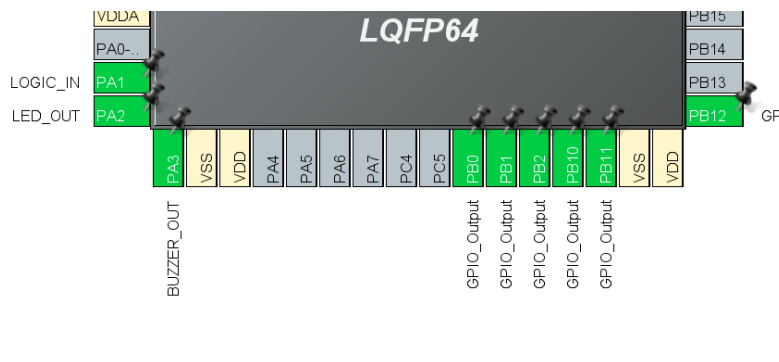
۱. تنظیمات سخت‌افزار در CubeMX:

- فرکانس کلاک اصلی میکرو را روی 15 MHz تنظیم کردیم.
- واحد TIM2 را برای ایجاد زمان‌بندی دقیق ۱ ثانیه‌ای فعال کردیم. طبق محاسبات، مقدار Prescaler برابر با 14999 و مقدار Counter Period (ARR) برابر با 999 قرار گرفت و وقفه عمومی تایمر (TIM2 global interrupt) را روشن کردیم.
- پین PA1 به عنوان ورودی (GPIO_Input) برای خواندن وضعیت لاجیک‌استیت تعریف شد.
- پین‌های PA2 و PA3 به عنوان خروجی برای LED و بازر ست شدند.
- پین‌های پورت B شامل PB0، PB1، PB2، PB10، PB11 و PB12 برای پین‌های RS و E طبق صورت سوال LCD در حالت خروجی قرار گرفتند.

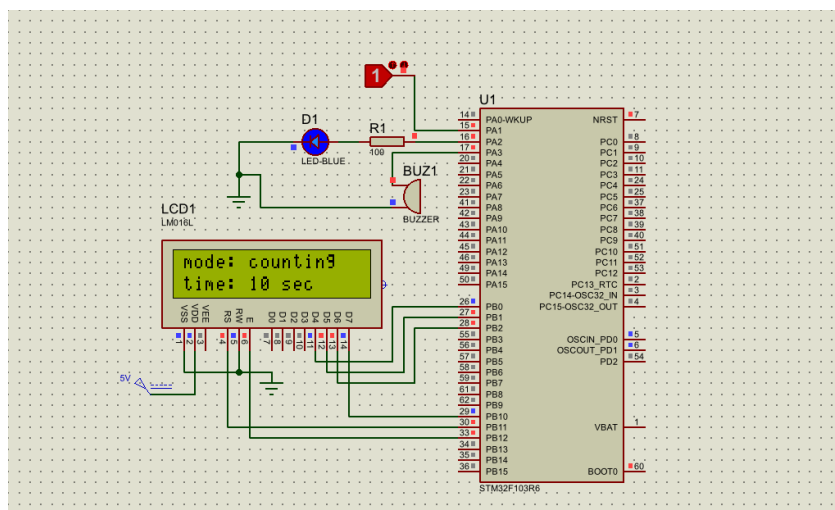
۲. منطق برنامه و توابع کد:

متغیر سراسری countdown_timer با مقدار اولیه ۱۵ تعریف شد.

- **داخل حلقه اصلی (while(1)):** وضعیت پین ورودی PA1 مدام چک می‌شود. اگر این پین صفر باشد، سیستم ریست شده، مقدار تایمر روی ۱۵ برمی‌گردد، خروجی‌ها خاموش می‌شوند و روی نمایشگر عبارت "mode: reset" چاپ می‌شود. اما اگر پین یک شود، شمارش معکوس شروع شده و مقدار ثانیه به همراه متن "mode: counting" روی سطر دوم LCD نشان داده می‌شود.
- **تابع وقفه تایمر (HAL_TIM_PeriodElapsedCallback):** چون تایمر روی ۱ ثانیه تنظیم شده، این تابع هر یک ثانیه یک‌بار اجرا می‌شود. اگر ورودی لاجیک‌استیت فعال باشد، یک واحد از countdown_timer کم می‌شود. تا زمانی که عدد بزرگتر از صفر است، پین‌های LED و بازر با دستور Toggle هر ثانیه چشمک و بوق می‌زنند. به محض اینکه شمارش به صفر رسید، روند کاهش زمان متوقف شده، خروجی‌های LED و بازر یکسره روشن می‌مانند و روی نمایشگر "counting is done" نوشته می‌شود.



شکل ۱۱: وضعیت پایه‌ها در فایل ioc



شکل ۱۲: شماتیک کامل مدار در نرم افزار پروتئوس

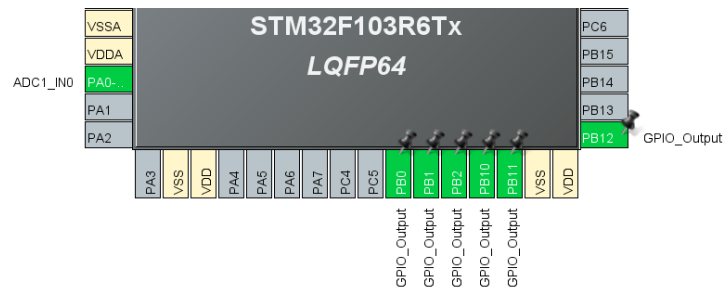
سوال ۴

بخش الف) بررسی واحد ADC و رزولوشن در STM32:

۱. واحد ADC در STM32
واحد ADC (مبدل آنالوگ به دیجیتال) وظیفه دارد ولتاژ پیوسته (بین ۰ تا ۳.۳ ولت) را به اعداد دیجیتال و قابل فهم برای پردازنده تبدیل کند.
۲. رزولوشن چند بیت است؟
رزولوشن پیش فرض و استاندارد در میکروکنترلر STM32F103 برابر با ۱۲ بیت است.
چون رزولوشن بالاتر به معنی دقت بیشتر در نمونه برداری است. یک دیتای ۱۲ بیتی می تواند باز ولتاژ آنالوگ را به $2^{12} = 4096$ قسمت یا پله دیجیتال (از عدد ۰ تا ۴۰۹۵) تقسیم کند.
۳. محاسبه Resolution Step:
وقتی ولتاژ مرجع ما $V_{ref} = 3.3V$ باشد، کوچک ترین تغییر ولتاژی که میکروکنترلر می تواند حس کند (دقت) برابر است با:

$$\text{Step} = \frac{3.3V}{4095} \approx 0.000805V \approx 0.8mV$$
در این سوال، پایه PA0 را که در سوال دوم برای خواندن پتانسیومتر استفاده کرده بودیم، اینجا نیز مجدداً استفاده می کنیم. بخش های ب، ج و د) منطق کنوپیسی، فرمول نویسی سنسورها و نمایش روی LCD:

۱. بخش USER CODE BEGIN PV (تعریف متغیر):
تنظیمات اولیه لدا در این بخش انجام شده است.
۲. بخش USER CODE BEGIN 2:
ساختار و پین های LCD با تابع Lcd_create مقاردهی اولیه شدند و نمایشگر با دستور Lcd_init راه اندازی شد و در آخر ADC با دستور HAL_ADC_Start شروع به کار می کند.
۳. نمونه برداری، فرمول نویسی و نمایش سنسورها (حلقه while):
در هر سیکل از حلقه (1) while، فرآیند خواندن مبدل و تفکیک سنسورها به این صورت انجام می شود:
 - محاسبه ولتاژ: ابتدا مقدار خام دیجیتال با دستور HAL_ADC_GetValue خوانده شده و در متغیر val_afc ذخیره می شود. سپس این عدد طبق فرمول استاندارد در 3.3 ضرب و بر 4095 تقسیم می شود تا ولتاژ دقیق پین در متغیر فلوت voltage قرار بگیرد.

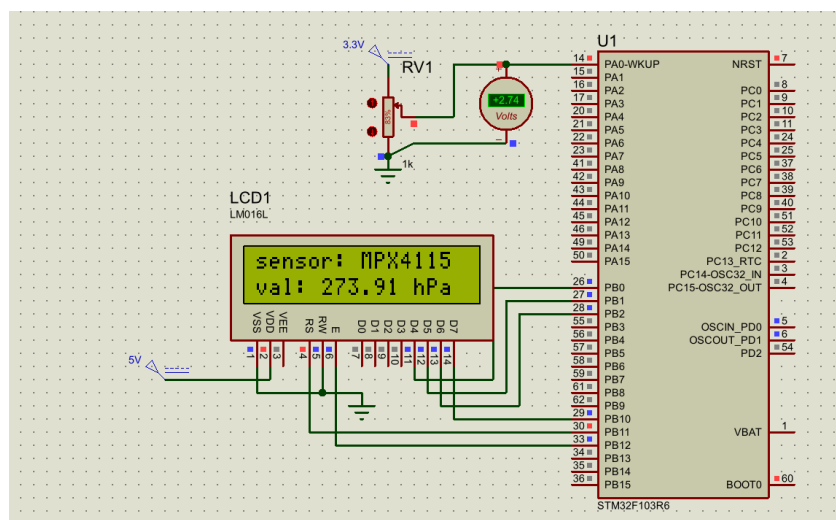


شکل ۱۳: وضعیت پایه‌ها در فایل ioc

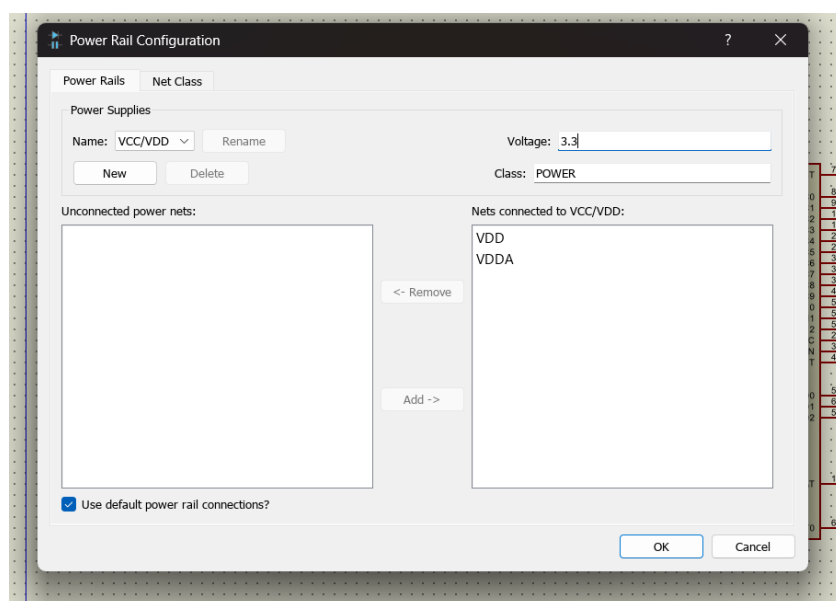
- **تفکیک بازه سنسورها:** یک ساختار شرطی if-else روی ولتاژ خطی ورودی قرار دادیم تا نوع سنسور و متغیر val_sen را مشخص کند:
 - **بازه 0 تا 1 ولت:** سنسور دما LM35 تشخیص داده شده و ولتاژ در ۱۰۰ ضرب می‌شود.
 - **بازه 1 تا 2 ولت:** سنسور رطوبت HIH-4000 تشخیص داده شده و ولتاژ در ۳.۳۳ ضرب می‌شود.
 - **بازه 2 تا 3 ولت:** سنسور فشار MPX4115 تشخیص داده شده و ولتاژ در ۱۰۰ ضرب می‌شود.
 - **بازه 3 تا 3.3 ولت:** سنسور گاز MQ-135 تشخیص داده شده و مقدار آن با ضرب ولتاژ در نسبت $\frac{100}{3.3}$ به دست می‌آید.
- **نمایش روی LCD:** به دلیل مشکل عدم پشتیبانی printf از متغیر float در میکروکنترلر، بخش اعشاری را به صورت دستی جدا کردیم. به این صورت که با کست کردن متغیر به (int)، جزء صحیح را برمی‌داریم و با فرمت %02d و عملیات ریاضی $((int)(val_sen * 100) \% 100)$ ، دو رقم اعشار اول را به صورت یک عدد صحیح مجزا استخراج می‌کنیم. در نهایت رشته‌های آماده شده در آرایه‌های firstline و secondline نمایشگر روی صفحه lcd چاپ می‌شوند.

اتصالات سخت‌افزاری و تنظیمات ولتاژ مرجع در پروتئوس:

- **رفع خطای ولتاژ کالیبراسیون:** از آنجا که پین‌های تغذیه آنالوگ میکروکنترلر در نرم‌افزار پروتئوس به صورت پیش‌فرض مخفی بوده و روی ۵ ولت قفل هستند (که باعث می‌شود نسبت خروجی ۵.۱ برابر خطا داشته باشد)، از بخش Power rail configuration، VCC را به ۳.۳ ولت تغییر دادیم و مشکل حل شد.



شکل ۱۴: شماتیک کامل مدار در نرم افزار پروتئوس



شکل ۱۵: رفع ولتاژ خطای کالیبراسیون